



Instructor: Date:

Learner: Regional Branch:

Program: Center:

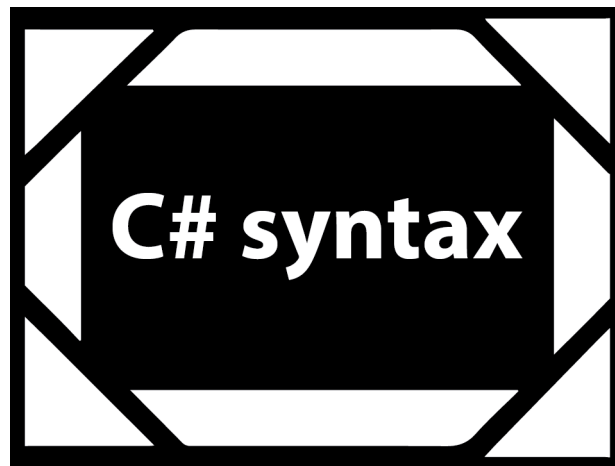
CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

Let's talk about C#: an object-oriented language.



Learning Outcome:

Talk about principles, concepts, and characteristics of general-purpose computer programming language in spoken and written situations.

Ask your partner:

- What are the most common commands in C#?
- What could happen if you mistype a word in C#?

Can Do Statements:

- I can talk about C# as an object oriented programming language.
- I can discuss and use basic syntax in C#.
- I can identify and use C# keywords.





Let's Explore!

Let's learn more about C# as an object-oriented language.



Instructions: Read the text below.

- Jeremy:** Why is C# an object-oriented language?
Lily: It is an object-oriented language because it's able to treat each class as an object instead of taking it as separate data and code.
- Jeremy:** What exactly is a class?
Lily: A class is a data structure that encompasses a set of data and behaviors; it's like a blueprint.
- Jeremy:** How do I start my C# program?
Lily: The first statement is "using System." Here you include the namespaces in the program. In the past, I would start by writing it into the code, but now, the program that I use writes this first part automatically.
- Jeremy:** What about the keywords?
Lily: There are keywords for different functions. For example, "bool," which introduces a Boolean logic of true and false. Another keyword is "if," which determines a condition that can take place inside classes. Also, you have identifiers for class, function, variables, etc.
- Jeremy:** This is very specific information and difficult to understand.
Lily: Don't worry! I also used to think that, but programming is about practice. Some months ago, I would spend a lot of time to practice programming at home. More than understanding what a variable is, you should remember the identifier used when writing a variable; it cannot have any spaces, start with a number or an underscore, or have any other symbols. When I started programming, I used to try to remember the theory; I didn't realize that you learn easier when practicing.
- Jeremy:** Okay, one mistake that I would make repeatedly was to name the variables with the names of a function.
- Lily:** That's something very important to take into account. Functions are a collection of statements that perform a specific task. If you give your variable the same name as a function, the computer doesn't know if it has to perform an action or not. On the other hand, a variable is a data member of a class used for storing data.

**Let's Practice!**

You'd better practice!

Instructions: Read the sentences and underline all the sentences that express a repeated action in the past.

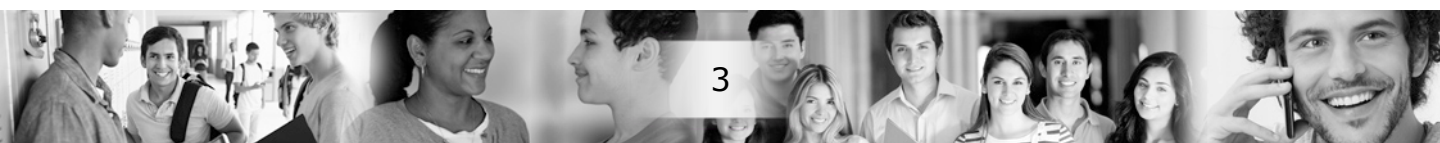
Programming is about practicing. I used to try and remember the theory, but I realized that it's easier to remember when you practice. A common mistake people would make was to name the functions the wrong way. I would name the variables with the names of a function, also. When this happens, the computer doesn't know if it has to perform an action or not. Unfortunately, there were no people to guide me during the process, and people would try to memorize everything. Now, it's much better because there are many instructions about it. I can't believe I used to think this was harder than it actually is.

**Let's Practice!**

Here are some features of C#'s syntax.

Instructions: Circle the words that best complete the sentences.

1. A class (encompasses / compass / passes) a set of data behaviors.
2. You always start your C# code with ("unique system" / "system using" / "using System").
3. The first statement provides the (spaces / namespaces / nicknames).
4. You use different (keywords / words / key holders) for all the functions.
5. (Identifiers / identities / identification) recognize class, functions, and variables.





Extension Activity

Making the concept of C# clear

Instructions: Complete the following steps. Fill in the Self-Assessment.

- In the web, find different examples of C# programming.
- Read and analyze the syntax of each example.
- In a poster, write the ones you consider are easy to replicate.
- Show your poster to a partner. Describe each text highlighting keywords, initial statement, variables, class, and functions.

Self-Assessment

Yes

No

- I find more information about C# programming. ☐ ☐
- I compare information from different texts. ☐ ☐
- I describe texts written on C# programming with their respective syntax. ☐ ☐



Target Words

- **collection** (noun): group, accumulation or selection
e.g. Functions are a collection of statements that perform a specific task.
- **treat** (verb): to deal with something in a certain way
e.g. It is an object-oriented language because it's able to treat each class as an object instead of taking it as separate data and code.
- **repeatedly** (adverb): happening once and again
e.g. Okay, one mistake that I would make repeatedly was to name the variables with the names of a function.
- **encompass** (verb): to have or include a determined set of things
e.g. A class is a data structure that encompasses a set of data and behaviors; it's like a blueprint.
- **identifier** (noun): a name that determines a specific object or a specific class of objects
e.g. Also, you have identifiers for class, function, variables, etc.
- **theory** (verb): a set of statements that explain a determined matter
e.g. I used to try to remember the theory; I didn't realize that you learn easier when practicing.



Learning Points

Used to and Would

We use **used to** to talk about past habits that we don't do anymore. We use **would** to talk about repeated actions in the past.

We use **used to** for repeated past events or actions that don't occur anymore.

Affirmative form structure: subject + **used to** + verb base form + complement

Example: *I **used to** start programming at 8:00 a. m. everyday.*

If we use this structure in the negative form, we use the auxiliary **did not** and the modal **use to** in its base form.

Negative form structure: Subject + **didn't use to** + complement

Example: *Laura **didn't use to** program with C#.*

As we can see in this example, we are talking about something that didn't happen before but happens now. It allows us to see that we use the **used to** structure to express difference between two recurrent events or actions tied to the path of time.

We use **would** for repeated actions in the past. The difference here is that we talk about only actions instead of habits.

Structure: Subject + **would** + verb + complement.

Example: *I **would** write the namespace, define the scope, and start with the classes.*

Note: You can't use this structure in the negative or interrogative forms.

Examples

- *I also used to think that, but programming is about practice.*
- *I used to try to remember the theory; I didn't realize that you learn easier when practicing.*
- *Some months ago, I would spend a lot of time to practice programming at home.*





Learning Points

C# Keywords.

There are some predefined keywords to the C# compiler. You shouldn't use these words as identifiers because it would create conflict in the code. If there's no option but to use one of these words, you use the symbol @ before the word.

Some other words have a contextual meaning. It means that they carry certain functions depending on the place they appear on.

The following charts have some reserved and some contextual keywords in C#. To look for the whole set of words, you can browse on the internet; some words are often generated and you ought to be up to date with them.

Reserved words

abstract	base	break	byte
catch	checked	const	continue
default	do	else	enum
explicit	false	fixed	float
foreach	if	in	in (generic modifier)
interface	is	long	namespace
null	operator	out (generic modifier)	override
private	public	ref	return
sealed	sizeof	static	string
switch	throw	try	typeof
ulong	unsafe	using	virtual
volatile			

Contextual keywords

add	ascending
dynamic	into
global	group
join	let